LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack dan Queue

Oleh:

Nayla Rizkia **NIM. 2510817120007**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Nayla Rizkia
NIM : 2510817120007

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN ..... **i**

DAFTAR ISI ..... **ii**

DAFTAR GAMBAR ..... **iii**

DAFTAR TABEL ..... **iv**

SOAL 1 ..... **1**

A. Source Code 1
B. Pembahasan 2

SOAL 2 ..... **5**

A. Source Code 6
B. Output Program 7
C. Pembahasan 7

SOAL 3 ..... **10**

A. Source Code 10
B. Pembahasan 11

SOAL 4 ..... **14**

A. Source Code 15
B. Output Program 16
C. Pembahasan 16

TAUTAN GIT **20**

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Jawaban Soal 2 7
Gambar 2 Screenshot Hasil Jawaban Soal 4 16

DAFTAR TABEL

Tabel 1 Source Code stack.cpp..... 1
Tabel 2 Source Code prob1.cpp 6
Tabel 3 Source Code queue.cpp 10
Tabel 4 Source Code prob2.cpp 15

SOAL 1

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1 Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = s->data - 1;
}

bool isEmpty(const Stack* s) {
    return s->top < s->data;
}

bool isFull(const Stack* s) {
    return s->top >= s->data + MAX - 1;
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw std::overflow_error("Stack penuh.");
    }
    s->top++;
    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
        throw std::underflow_error("Stack kosong.");
    }
    s->top--;
}
```

```

int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw std::underflow_error("Stack kosong.");
    }
    return *(s->top);
}
```

B. Pembahasan

`#include "stack.h"` mengimpor file header yang berisi definisi stack struct, yang mencakup pointer `top` dan array data, serta deklarasi fungsi yang akan diimplementasikan.

`#include <stdexcept>` mengimpor pustaka C++ standar untuk menggunakan mekanisme penanganan pengecualian, seperti `std::overflow_error` dan `std::underflow_error` saat terjadi kasus tidak valid.

Fungsi `init` bertujuan untuk menginisialisasi kondisi awal dari `Stack`. Karena `top` bertipe pointer ke integer (`int*`), pointer ini diatur untuk menunjuk ke "alamat sebelum elemen pertama" array data. Pada kondisi awal (kosong), pointer `top` tidak boleh menunjuk ke indeks elemen yang valid. Dengan menempatkannya di `s->data - 1`, saat operasi `push` pertama kali dipanggil, di mana pointer akan di-increment, pointer akan bergerak maju dan menunjuk ke `s->data[0]`, lokasi yang tepat untuk elemen pertama. Kompleksitas:

Waktu: O(1) karena hanya melakukan operasi aritmatika dengan pointer dan assign.

Ruang: O(1) karena tidak ada alokasi memori tambahan.

Fungsi `isEmpty` menentukan apakah stack dalam kondisi kosong. Jika alamat memori yang ditunjuk oleh pointer `top` lebih kecil atau berada sebelum alamat memori awal dari array data, fungsi akan mengembalikan `true`. Berdasarkan fungsi `init`, pointer `top` berada di posisi `s->data - 1` saat stack kosong. Jika `top` lebih kecil dari `s->data`, itu menunjukkan secara logis bahwa tidak ada data yang disimpan di dalam stack. Kompleksitas:

Waktu: O(1) karena hanya ada satu operasi perbandingan.

Ruang: O(1).

Fungsi `isFull` mengecek apakah kapasitas maksimal stack sudah tercapai. Jika pointer `top` berada pada (atau melewati) alamat elemen terakhir array, fungsi mengembalikan true. Kapasitas maksimum didefinisikan sebagai `MAX`, sehingga alamat elemen terakhir adalah memori awal array ditambah `MAX - 1`. Fungsi ini membandingkan alamat memori elemen teratas, atau `top`, dengan alamat memori batas maksimal array. Jika alamat teratas mencapai atau melewati batas alamat maksimal, itu menunjukkan secara matematis bahwa tidak ada lagi ruang memori kosong di stack. Kompleksitas:

Waktu: O(1) karena hanya ada satu operasi penjumlahan dan satu operasi perbandingan.

Ruang: O(1).

Fungsi `push` memanfaatkan `isFull`(s) untuk memvalidasi kapasitas stack. Program akan melempar exception `std::overflow_error` jika stack telah mencapai batas maksimalnya. Namun, jika masih ada ruang, pointer `top` akan dinaikkan satu posisi (`s->top++`). Proses dereferensi kemudian akan digunakan untuk mengisi nilai baru. Penanganan exception memastikan program tidak mengalami segmentation fault akibat mengakses memori di luar batas array. Untuk memastikan bahwa elemen ditambahkan secara berurutan, urutan `s->top++` dilakukan sebelum pengisian nilai (assignment). Elemen paling atas (LIFO) selalu ditunjukkan sebagai elemen paling atas. Kompleksitas:

Waktu: O(1) karena proses validasi dan penugasan berlangsung secara bersamaan.

Ruang: O(1).

Fungsi `pop` memvalidasi kondisi stack dengan `isEmpty(s)`. Untuk mencegah kesalahan akses data, program akan memicu exception `std::underflow_error` jika stack kosong. Jika data tersedia, fungsi akan menghapus elemen teratas secara logis dengan menggeser pointer `top` mundur satu langkah (`s->top--`). Dengan menurunkan pointer `top`, elemen yang secara logis berada di atas sudah terhapus dan tidak dianggap

lagi. Data lama akan ditimpa atau dioverwrite dengan sendirinya jika ada pemanggilan `push` baru di masa depan. Kompleksitas:

Waktu: O(1) karena pengujian dan pengurangan memori (pengurangan) terjadi secara instan atau konstan.

Ruang: O(1).

Fungsi `peek` akan melempar exception `std::underflow_error` dan memvalidasi isi stack menggunakan `isEmpty(s)` jika tidak ada data yang ditemukan. Fungsi dapat mengembalikan nilai pada posisi teratas dengan melakukan dereferensi langsung pada pointer `top` (`*(s->top)`) dalam kasus di mana stack berisi elemen. `peek` memungkinkan untuk melihat elemen teratas tanpa mengubah isi. Selain itu, fungsi ini terbukti aman secara memori karena tidak merubah letak pointer `top` atau isi array, tetapi masih berhasil mengembalikan informasi nilai teratas. Kompleksitas:

Waktu: O(1) karena pengecekan dan dereferensi berlangsung secara konsisten.

Ruang: O(1) karena tidak menghasilkan salinan data baru selain return value normal.

SOAL 2

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerjayang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling barudimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir. Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- A_i adalah nilai bilangan pada input
- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	**Output**
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Tabel 2 Source Code prob1.cpp

```
#include "stack.h"
#include <iostream>
#include <string>
#include <stdexcept>

using namespace std;

int main() {
    int n;
    cin >> n;

    Stack s;
    init(&s);

    for (int i = 0; i < n; i++) {
        string token;
```

```
        cin >> token;

        if (token == "+" || token == "-" || token == "*" || token == "/") {
            int angka2 = peek(&s);
            pop(&s);
            int angka1 = peek(&s);
            pop(&s);

            int hasil = 0;

            if (token == "+") {
                hasil = angka1 + angka2;
            } else if (token == "-") {
                hasil = angka1 - angka2;
            } else if (token == "*") {
                hasil = angka1 * angka2;
            } else if (token == "/") {
                hasil = angka1 / angka2;
            }
            push(&s, hasil);
        } else {
            int angka = stoi(token);
            push(&s, angka);
        }
    }

    cout << peek(&s) << endl;

    return 0;
}
```

B. Output Program

```
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> g++ stack.cpp prob1.cpp -o prob1
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> .\prob1.exe
3
2 3 +
5
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> g++ stack.cpp prob1.cpp -o prob1
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> .\prob1.exe
9
5 1 2 + 4 * + 3 -
14
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> g++ stack.cpp prob1.cpp -o prob1
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> .\prob1.exe
5
10 2 / 3 *
15
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main>
```

Gambar 1 Screenshot Hasil Jawaban Soal 2

C. Pembahasan

`#include "stack.h"` memanggil file header yang berisi definisi Stack dan fungsi-fungsinya.

`#include <iostream>` memuat library standar C++ untuk input dan output (`cin` dan `cout`).

`#include <string>` memuat tipe data string untuk menangani input teks.

`#include <stdexcept>` memuat library penanganan error bawaan C++.

`using namespace std;` memungkinkan penulisan perintah standar tanpa perlu prefix `std::`.

`int main()` merupakan fungsi utama di mana eksekusi program dimulai.

`int n; cin >> n;` mendeklarasikan variabel integer `n` dan menerima input dari pengguna.

`Stack s;` membuat sebuah instance dari struct Stack dengan nama `s`.

`init(&s);` memanggil fungsi inisialisasi dengan memberikan alamat memori atau referensi dari stack `s`.

`for (int i = 0; i < n; i++)` adalah struktur perulangan yang akan berjalan sebanyak `n` kali.

`string token; cin >> token;` menyediakan variabel penyimpan teks dan mengambil input satu kata atau simbol dari pengguna per iterasinya.

`if (token == ...)` mengecek apakah string input sama dengan salah satu dari empat simbol aritmatika dasar. Percabangan `if-else` bersarang melakukan operasi matematika sesuai dengan simbol string yang diterima.

`peek(&s)` dan `pop(&s)` memanggil fungsi stack untuk melihat nilai teratas, lalu menghapusnya dari tumpukan.

`push(&s, hasil)` memasukkan kembali hasil perhitungan ke dalam stack.

Perulangan ini memproses operator dengan mengambil dua angka teratas dari stack sesuai dengan prinsip LIFO Program menghitung hasil operasi antara kedua angka dan memasukkannya kembali ke dalam stack untuk diproses pada tahap kalkulasi berikutnya.

`else` menangkap semua kondisi selain empat operator sebelumnya.

`stoi(…)` adalah fungsi C++ yang dapat mengubah teks yang berisi angka menjadi nilai numerik asli.

Program menganggap `token` yang dievaluasi sebagai operand atau angka jika bukan operator aritmatika. Setelah diubah menjadi integer, teks angka dimasukkan secara langsung ke dalam stack dan menunggu giliran untuk dihitung saat operator muncul.

`cout << peek(&s) << endl;` mencetak nilai elemen teratas dari stack `s` ke layar terminal, lalu mencetak baris baru.

`return 0;` menandakan program selesai dengan sukses.

Time Limit	1.0 s
Memory Limit	64 MB
Kompleksitas Waktu	O(n) Program memproses input secara linier. Algoritma dengan kompleksitas O(n) hanya akan dapat melakukan sekitar ratusan operasi dasar, karena batas maksimal `n` hanyalah 100. Komputer kontemporer dapat melakukan puluhan juta operasi dalam satu detik.
Kompleksitas Ruang	O(n) Array data berkapasitas 100 elemen integer yang digunakan hanyalah sekitar 400 byte, karena satu integer biasa berukuran 4 byte, jadi array ini hanya memakan sekitar 400 byte.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisiskompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3 Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = q->data;
    q->rear = q->data - 1;
}

bool isEmpty(const Queue* q) {
    return q->front > q->rear;
}

bool isFull(const Queue* q) {
    return (q->rear - q->front + 1) >= MAX;
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
        throw std::overflow_error("Queue penuh.");
    }
    if (q->rear >= q->data + MAX - 1) {
        int jumlah_elemen = q->rear - q->front + 1;
        for (int i = 0; i < jumlah_elemen; i++) {
            *(q->data + i) = *(q->front + i);
        }
        q->front = q->data;
        q->rear = q->data + jumlah_elemen - 1;
    }
```

```
    q->rear++;
    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue kosong.");
    }
    q->front++;

    if (q->front > q->rear) {
        q->front = q->data;
        q->rear = q->data - 1;
    }
}

int front(const Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue kosong.");
    }
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue kosong.");
    }
    return *(q->rear);
}
```

B. Pembahasan

`#include "queue.h"` mengimpor definisi struct Queue yang berisi array data, pointer `front`, dan `rear`, serta deklarasi fungsinya.

`#include <stdexcept>` mengimpor pustaka C++ standar untuk menggunakan mekanisme penanganan pengecualian, seperti `std::overflow_error` dan `std::underflow_error` saat terjadi kasus tidak valid.

Fungsi `init` menginisialisasi keadaan awal antrean kosong. `front` berada di garis awal dengan indeks 0, sementara `rear` berada di belakang garis awal. Logika ini sejalan dengan metode pengisian yang akan datang. Setelah data pertama dikirim, `rear` akan maju satu langkah hingga posisinya sejajar dengan `front` di indeks 0. Kompleksitas:

Waktu: O(1) karena operasi penugasan alamat pointer hanya ada dua.

Ruang: O(1).

Fungsi `isEmpty` menentukan apakah queue dalam kondisi kosong. Saat dimulai, `front` berada di data dan `rear` di `data - 1`, artinya `front` > `rear` (kosong). Ketika data masuk, `rear` akan maju sehingga `rear >= front`. Jika data dikeluarkan (dequeue) sampai habis, `front` akan terus maju melewati `rear`, mengembalikan kondisi ke `front` > `rear`, yang menandakan antrean kembali kosong. Kompleksitas:

Waktu: O(1) karena ada satu operasi komparatif.

Ruang: O(1).

Fungsi `isFull` mengecek apakah total elemen yang sedang berada di dalam antrean sudah menyamai kapasitas array. Meskipun queue memiliki ujung akhir (`rear`) yang sudah mentok di batas array, ujung depan (`front`) memiliki memori kosong karena ada elemen yang sudah di-dequeue. Dengan menggunakan rumus (`rear` – `front + 1`), fungsi ini dapat menghitung total elemen aktual dan menunjukkan kapasitas sebenarnya. Kompleksitas:

Waktu: O(1).

Ruang: O(1).

Fungsi `enqueue` diawali dengan validasi `isFull(q)` yang akan melempar exception `overflow_error` jika kapasitas antrean sudah maksimal. Apabila pointer `rear` telah mencapai batas akhir memori tetapi antrean belum penuh, fungsi akan melakukan shifting dengan mengembalikan seluruh elemen ke indeks awal (`q->data`) dan mengatur ulang kedua posisi `rear` dan `front`. Setelah ruang tersedia di tempat yang tepat, pointer `rear` dimajukan satu langkah untuk memasukkan nilai baru melalui operasi dereferensi. Algoritma pergeseran (shifting) ini mencegah hal-hal yang terlihat seolah-olah array penuh karena `rear` berada di ujung array, meskipun di depan ada slot kosong. Kompleksitas:

Waktu: O(n) pada worst-case, yang terjadi ketika memori mentok dan perlu menggeser elemen sebanyak `n`, tetapi O(1) adalah best-case, yang terjadi ketika `rear` belum mentok.

Ruang: O(1) karena perpindahan dilakukan di memori yang sama tanpa array sementara.

Fungsi `front` dan `back` digunakan untuk "mengintip" siapa yang berada di urutan terdepan (yang paling siap dikeluarkan) dan paling belakang (yang baru saja masuk), tanpa mengubah posisi antrean sama sekali. Kedua fungsi ini aman karena tidak mengubah pointer pembatas atau data array. Kompleksitas:

Waktu: O(1).

Ruang: O(1).

SOAL 4

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angkayang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- A_i: elemen array

Format Input

n k

A1 A2 … An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemendalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	**Output**
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Tabel 4 Source Code prob2.cpp

```
#include "queue.h"
#include <iostream>

using namespace std;

int main() {
    int n, k;
    cin >> n >> k;

    int arr[105];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

   if (k <= 0 || k > n) {
        return 0;
    }

    Queue q;
    init(&q);

    int current_sum = 0;


    for (int i = 0; i < k; i++) {
        enqueue(&q, arr[i]);
```

```
        current_sum += arr[i];
    }

    cout << current_sum;

    for (int i = k; i < n; i++) {
        current_sum -= front(&q);
        dequeue(&q);
        enqueue(&q, arr[i]);
        current_sum += arr[i];

        cout << " " << current_sum;
    }

    cout << endl;

    return 0;
}
```

B. Output Program

```
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> g++ queue.cpp prob2.cpp -o prob2
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> .\prob2.exe
5 3
1 2 3 4 5
6 9 12
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> g++ queue.cpp prob2.cpp -o prob2
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> .\prob2.exe
5 5
1 2 3 4 5
15
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> g++ queue.cpp prob2.cpp -o prob2
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main> .\prob2.exe
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
PS D:\w\semester 2\prak algo\modul 2\module-2-stack-and-queue-alyanriz-main>
```

Gambar 2 Screenshot Hasil Jawaban Soal 4

C. Pembahasan

`#include "queue.h"` memanggil file header yang berisi definisi struktur data Queue dan fungsi-fungsinya (seperti `enqueue`, `dequeue`, dan lain-lain).

`#include <iostream>` memuat library standar C++ untuk input dan output (`cin` dan `cout`).

`using namespace std;` memungkinkan penulisan perintah standar tanpa perlu prefix `std::`.

`int main()` merupakan fungsi utama di mana eksekusi program dimulai.

`int n, k; cin >> n >> k;` adalah deklarasi dan pengambilan input untuk variabel `n` (total elemen array) dan `k` (ukuran jendela atau sub-array).

`int arr[105];` mendeklarasikan statik array berukuran 105.

`for (...) { cin >> arr[i]; }` merupakan perulangan untuk mengisi array tersebut dengan angka-angka dari input pengguna.

`if (k <= 0 || k > n)` adalah blok percabangan kondisional. Program langsung dihentikan dengan `return 0` jika nilai `k` tidak masuk akal (kurang dari sama dengan 0, atau lebih besar dari total elemen `n`).

`Queue q; init(&q);` membuat instansiasi antrean baru bernama `q` dan menyiapkan pointernya ke kondisi awal (kosong).

`int current_sum = 0;` merupakan variabel penyimpan total penjumlahan sementara.

`for (int i = 0; i < k; i++)` adalah perulangan yang berjalan dari indeks 0 hingga tepat sebelum ukuran `k`.

`enqueue(...)` dan `current_sum += ...` memasukkan data ke antrean sekaligus menambahkannya ke variabel `current_sum`.

`cout << current_sum`; mencetak hasil jumlahan dari jendela pertama ke layar.

Program membangun "jendela" pertama, yang berukuran sama dengan elemen pertama. Antrean (yang berfungsi sebagai bingkai jendela) mengumpulkan angka-angka tersebut. Kelompok pertama ini mencetak hasil mereka langsung ke layar.

`for (int i = k; i < n; i++)` adalah perulangan yang melanjutkan indeks dari `k` (elemen setelah jendela pertama) hingga akhir array `n`.

`current_sum -= front(&q);` mengurangi total jumlahan saat ini dengan nilai dari elemen terdepan di antrean.

`dequeue(&q);` membuang elemen terdepan tersebut dari antrean.

`enqueue(&q, arr[i]);` memasukkan elemen baru (elemen ke-`i`) dari ujung kanan ke dalam antrean.

`current_sum += arr[i];` menambahkan elemen baru tersebut ke dalam total jumlahan.

`cout << " " << current_sum;` mencetak hasil yang baru dihitung, dipisahkan dengan spasi.

Ketika jendela bergerak ke kanan, elemen di posisi paling kiri dikeluarkan dari Queue dengan operasi `front()` untuk mengurangi dari `current_sum` dan kemudian dihapus melalui `dequeue()`. Secara bersamaan, elemen baru di posisi paling kanan dimasukkan ke dalam antrean dengan `enqueue()`, dan nilainya ditambahkan ke `current_sum`. Teknik jendela bergulir ini mengoptimalkan efisiensi waktu menjadi O(n) karena perhitungan tidak perlu diulang dari awal. Sebaliknya, nilai diperbarui setiap pergeseran dengan kompleksitas O(1).

Time Limit	1.0 s
Memory Limit	64 MB
Kompleksitas Waktu	O(n) Meskipun ada proses pergeseran jendela dari indeks `k` hingga n, setiap elemen dapat masuk (`enqueue`) satu kali dan keluar (`dequeue`) maksimal satu kali, total operasi berbanding lurus dengan nilai `n`.
Kompleksitas Ruang	O(n) Deklarasi array statis `arr[105]` dan array `data[MAX]`, yang masing-masing memiliki maksimal memori 100, di dalam struktur Queue, memberikan alokasi memori terbesar

	untuk program ini. Total memori yang dialokasikan untuk kedua array ini tidak lebih dari 1 Kilobyte.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-alyanriz/tree/main